

0826 Answer | 并发约束：把交错执行压成可证明的顺序

原题 15 题 · 0825 进程与调度 · 能力账本

已会辨认线程共享变量与进程就绪/阻塞；本节点从“谁先读、谁后写、等待什么条件”出发限制交错。15 题均已核准。

15 题考场入口

题	第一笔	答案
01	信号量正数是现有空闲数，负数才数等待者	B
02	flag 表想进，turn 冲突让给对方	D
03	两次读写可丢更新，最终 0/1/2	C
04	临界区不等于 CPU 永不调度	C
05	三者各差一台时最多占 9，需第 10 台推进	B
06	TSL 循环是忙等，不主动放弃 CPU	B
07	同一进程的 $x+=1$ 与 $x+=2$ 会冲突	C
08	管程还可做条件同步	A
09	condition.wait 阻塞调用者并入等待队列	D
10	信号量 P 等待可阻塞让权	C
11	I/O 完成、退出临界区可唤醒；片尽仅就绪轮换	C
12	互斥、空闲可进、有限等待；让权非必需	C
13	同进程线程共地址空间及 fd，不共栈	B
14	队尾每轮等九个 5ms，自己五片，250ms	C
15	最少页框要容纳指令可能同时访问的页面	D

01 | 2010-25：先看信号量符号，别把初值与当前值相减当等待数

25. 设与某资源关联的信号量初值为 3，当前值为 1。若 M 表示该资源的可用个数， N 表示等待该资源的进程数，则 M 、 N 分别是_____。

- A. 0、1 B. 1、0 C. 1、2 D. 2、0

资源信号量初值 3，当前值 $1 > 0$ ，表示现在还可用 1 个资源， $M=1$ ；此时没有进程因该资源等待， $N=0$ ，选 B。第一笔写 $S > 0 \rightarrow$ 可用 S 、等待 0，若 $S < 0$ 才用 $-S$ 解释等待个数。

初值 3 到当前 1 说明了什么

在典型 P/V 信号量模型里，差 2 说明当前已有两个资源单位被占用或净申请；它不是“两个进程正在等”。等待的判据是申请后资源不够，信号量记为负值。

02 | 2010-27：Peterson 代码要看同时申请时最后谁把 turn 写给谁

27. 进程 P_0 和 P_1 的共享变量定义及其初值为：

boolean flag[2];

int turn=0;

flag[0]=FALSE; flag[1]=FALSE;

若进程 P_0 和 P_1 访问临界资源的类 C 伪代码实现如下：

```
void P0()    //进程 P0
{
    while (TRUE)
    {
        flag[0]=TRUE; turn=1;
        while(flag[1] && (turn==1));
        临界区;
        flag[0]=FALSE;
    }
}
```

```
void P1()    //进程 P1
{
    while (TRUE)
    {
        flag[1]=TRUE; turn=0;
        while(flag[0] && (turn==0));
        临界区;
        flag[1]=FALSE;
    }
}
```

则并发执行进程 P_0 和 P_1 时产生的情形是_____。

- A. 不能保证进程互斥进入临界区，会出现“饥饿”现象
B. 不能保证进程互斥进入临界区，不会出现“饥饿”现象
C. 能保证进程互斥进入临界区，会出现“饥饿”现象
D. 能保证进程互斥进入临界区，不会出现“饥饿”现象

两个进程先各置 $flag[i]=TRUE$ 表示想进，再把 turn 写成对方编号。若都想进，最终 turn 只能是 0 或 1：一方在 $while(flag[other] \&\& turn==other)$ 等，另一方可进入；退出后清自己的 flag，让等待者继续。因此既能互斥进入也不会因这段竞争发生饥饿，选 D。第一笔把“两个 flag 都真”代入两个 while，看 turn 的最终值。

逐位试 turn=0

turn=0 时 P1 的条件 `flag[0] && turn==0` 为真，P1 等；P0 的条件 `flag[1] && turn==1` 为假，P0 进。若 turn=1 则相反。先进去者退出清 flag，等待者的条件变假。这个证明依赖题目通常假设的原子读写与顺序可见性，实际弱内存机器需相应同步原语。

03 | 2011-32：加一减一不是原子动作，列最后写回者读过什么

32. 有两个并发执行的进程 P₁ 和 P₂，共享初值为 1 的变量 x。P₁ 对 x 加 1，P₂ 对 x 减 1。加 1 和减 1 操作的指令序列分别如下所示。

<pre>//加 1 操作 load R1, x //取 x 到寄存器 R1 中 inc R1 store x, R1 //将 R1 的内容存入 x</pre>	<pre>//减 1 操作 load R2, x dec R2 store x, R2</pre>
--	---

两个操作完成后，x 的值_____。

- | | |
|----------------|------------------|
| A. 可能为-1 或 3 | B. 只能为 1 |
| C. 可能为 0、1 或 2 | D. 可能为-1、0、1 或 2 |

x 初值 1；每个操作均拆成 load→inc/dec→store。先完整加后完整减或反过来得 1。若两者都先读到 1，再分别写 2 和 0，最后写者决定结果 2 或 0。可能 0、1、2，选 C。第一笔抓共享内存上的两次 load 与两次 store；寄存器计算不直接改变 x。

为什么不会是 -1 或 3

减一者读到的共享值只能是最初 1 或加一者写出的 2，所以它只会写 0 或 1；加一者读到的只能是最初 1 或减一者写出的 0，只会写 2 或 1。最终一次写回来自其中一个线程，所以范围只有 0、1、2。

04 | 2012-30：临界区保护共享数据，不自动禁止调度

30. 若某单处理器多进程系统中有多个就绪态进程，则下列关于处理机调度的叙述中，错误的是_____。

- A. 在进程结束时能进行处理机调度
- B. 创建新进程后能进行处理机调度
- C. 在进程处于临界区时不能进行处理机调度
- D. 在系统调用完成并返回用户态时能进行处理机调度

进程结束、新进程建立后或系统调用返回前都可能出现调度机会，A/B/D 可成立。进程正在临界区时也可能因时间片到、抢占等原因离开 CPU；互斥约束的是另一进程不能同时进入同一资源的临界区，并非 OS 不能调度，C 错，选 C。第一笔分开“谁持有锁/进入临界区”与“谁在 CPU 上运行”。

被换下时锁还在怎么办

其他线程到该临界区要等待持有者释放；原线程以后重新调度并完成临界区后再退出。某些内核临界区可暂禁抢占，那是特定实现策略，题句笼统说所有进程在临界区不能调度，不能成立。

05 | 2014-24：构造所有人都差最后一台的最坏占用

24. 某系统有 n 台互斥使用的同类设备，三个并发进程分别需要 3、4、5 台设备，可确保系统不发生死锁的设备数 n 最小为_____。

- A. 9 B. 10 C. 11 D. 12

三个进程最大需求分别 3、4、5。要全卡住，可让它们各已占到还差 1 台：分别占 2、3、4，共 9 台，却谁也完不成、归还设备。若有 10 台，该最坏占用后至少还剩 1，能给某一进程凑够最大需求，它完成后归还全部设备，后面也能推进。所以保证不死锁的最小 $n=10$ ，选 B。第一笔用 $\Sigma(\text{最大需求}-1)+1$ 。

为什么 9 是真正反例

分配给三者 $2+3+4=9$ ，每者都再请求一台且互不释放，系统无空闲，形成等待。 $n=10$ 时对任何“所有进程都无法完成”的状态，每者至多持有最大需求减 1，合计最多 9；多出的至少一台可以让某者完成。此处同类设备可互换，且进程完成会释放所持设备。

01—05 收束

看到同步题先标共享量、原子动作、等待条件、谁可继续；信号量先看符号，软件互斥先代入双方都申请，交错计算先看最后写者。死锁资源题构造“人人差一份”的极端，再加一份保证有进程能结束。讲解完成度当前 5/15，独立熟练度待测。

06 | 2016-27：TSL 能互斥，但等待者会反复占 CPU 查锁

27. 使用 TSL (Test and Set Lock) 指令实现进程互斥的伪代码如下所示。

```
do{
    ...
    while(TSL(&lock)) ;
    critical section;
    lock=FALSE;
    ...
} while(TRUE);
```

下列与该实现机制相关的叙述中，正确的是_____。

- A. 退出临界区的进程负责唤醒阻塞态进程
B. 等待进入临界区的进程不会主动放弃 CPU
C. 上述伪代码满足“让权等待”的同步准则
D. while(TSL(&lock))语句应在关中断状态下执行

TSL(&lock) 原子地读旧锁值并置锁；旧值为真，while 就继续循环检查，不会主动睡眠或让出 CPU，B 对，选 B。退出者只把 $\text{lock}=\text{FALSE}$ ，没有唤醒队列中的阻塞进程，A 错；忙等不满足让权等待，C 错；TSL 本身的读改写须原子，不要求整个 while 都关中断，D 错。第一笔观察等待循环里有没有阻塞调用。

在单核上尤其要辨“CPU 调度”与“主动放弃”

忙等进程仍可能被时间片调度器强制换下，但它自身没有把 CPU 让给被锁挡住以外的工作。锁持有者仍需获调度、执行完临界区才能释放；把“会被抢占”当作“满足让权等待”会误选。

07 | 2016-30：先划进程地址空间，再找同一个共享变量的写写冲突

30. 进程 P_1 和 P_2 均包含并发执行的线程，部分伪代码描述如下所示。

<pre>//进程 P₁ int x = 0; Thread1() { int a; a = 1; x += 1; } Thread2() { int a; a = 2; x += 2; }</pre>	<pre>//进程 P₂ int x = 0; Thread3() { int a; a = x; x += 3; } Thread4() { int b; b = x; x += 4; }</pre>
---	---

下列选项中，需要互斥执行的操作是_____。

- | | |
|------------------------|------------------------|
| A. $a = 1$ 与 $a = 2$ | B. $a = x$ 与 $b = x$ |
| C. $x += 1$ 与 $x += 2$ | D. $x += 1$ 与 $x += 3$ |

P_1 内 Thread1、Thread2 共享 P_1 的 x ，各做 $x+=1$ 与 $x+=2$ ，非原子读改写交错会丢失更新，需要互斥，选 C。 P_2 自己另有一个 x ，名字相同不等于与 P_1 共享； $x+=1$ 与 P_2 的 $x+=3$ 不构成同一地址的冲突。各线程局部 a 也独立。第一笔在变量旁标 $P_1.x$ 或 $P_2.x$ 。

为什么两个只读操作不用互斥

$a=x$ 、 $b=x$ 即使同时发生，只读不会彼此覆盖；本题的危险是同一 $P_1.x$ 上两个写操作都先读旧值再写新值。互斥的对象是共享状态的冲突访问，不是凡有两个线程就一律上锁。

32. 下列关于管程的叙述中，错误的是_____。

- A. 管程只能用于实现进程的互斥
- B. 管程是由编程语言支持的进程同步机制
- C. 任何时候只能有一个进程在管程中执行
- D. 管程中定义的变量只能被管程内的过程访问

题 33~41 均依据题 33~41 图回答。

题问错误。管程的受控入口可保证同一时刻只有一个进程在其中执行 C，对内部变量限制外部直接访问 D，语言/运行时可支持这一同步结构 B。A 说“只能用于互斥”，却漏了管程内条件变量的等待/通知可表达条件同步，选 A。第一笔寻找这个“只能”能否被条件等待反驳。

互斥与同步各解决什么

互斥防止两者同时改共享状态；条件同步要求“状态未满足就先等，满足后再继续”，如缓冲区空时消费者等待。管程把两件事封装在受控操作里，因此不只是锁。

28. 若 x 是管程内的条件变量，则当进程执行 x.wait()时所做的工作是_____。

- A. 实现对变量 x 的互斥访问
- B. 唤醒一个在 x 上阻塞的进程
- C. 根据 x 的值判断该进程是否进入阻塞状态
- D. 阻塞该进程，并将之插入 x 的阻塞队列中

进程执行 x.wait()，表示当前条件尚未满足：它释放管程占有、阻塞自己并加入条件变量 x 的等待队列，选 D。B 的“唤醒一个”等待者是 signal() 的方向；A 不是单纯互斥；C 不是读一个普通数值来判断。第一笔在 wait 旁写“谁调用谁睡”，在 signal 旁写“唤醒别人”。

为什么等待时必须让出管程

若等待者仍独占管程，别的进程进不来改变条件并发信号，系统无法推进。条件等待会在相应语义下释放管程互斥，唤醒并重新获进入资格后再继续；具体 Hoare/Mesa 调度细节此题不需展开。

10 | 2018-32：让权等待问“等不到时是否睡眠”

32. 下列同步机制中，可以实现让权等待的是_____。

- A. Peterson 方法 B. swap 指令 C. 信号量方法 D. TestAndSet 指令

Peterson、swap 与 TestAndSet 都可在等待条件未满足时自旋检查，继续消耗 CPU；信号量的 P 操作在资源不足时可把进程置阻塞、让出处理器，满足让权等待，选 C。第一笔查看等待分支执行的是“反复读条件”还是“入队阻塞”。

与 01、06 接合

01 的资源信号量变负可表示等待者；其典型实现把等待进程挂在队列中并调度别人。06 的 TSL 仅循环读锁，没有这步睡眠队列。二者都可用于互斥，但等待成本不同。

06—10 收束

同步题先把共享变量的归属画清，再看竞争访问；等待代码若是 while 反复检查就是忙等，P 或 condition.wait 在条件不满足时可阻塞让权。管程把互斥入口与条件同步一起封装。不要把“能互斥”自动推为“无饥饿”或“让权等待”。

11 | 2019-24：唤醒是“阻塞→就绪”，时间片用完不在这条边上

24. 下列选项中，可能会将进程唤醒的事件是_____。

- I. I/O 结束 II. 某进程退出临界区 III. 当前进程的时间片用完
A. 仅 I B. 仅 III C. 仅 I、II D. I、II、III

I/O 结束可让等 I/O 的进程就绪 I；持有临界资源者退出临界区可通知等锁者 II。当前进程时间片用尽 III 只把它自己从执行送回就绪，不是唤醒一个原本阻塞的进程。因此 I、II，选 C。第一笔给“被唤醒者”画出原状态：它先前必须在等某个条件。

不是每次退出临界区都一定有被唤醒者

题问“可能”，若确有等待该锁/条件的进程，释放可使其获得进入机会；无等待者则没有唤醒。时间片换人可能让已就绪者运行，但“被调度上 CPU”与“从阻塞转就绪”不是同一动作。

12 | 2020-32：互斥机制的三项必须条件，不强制让权等待

32. 下列准则中，实现临界区互斥机制必须遵循的是（ ）。

- I. 两个进程不能同时进入临界区
 - II. 允许进程访问空闲的临界资源
 - III. 进程等待进入临界区的时间是有限的
 - IV. 不能进入临界区的执行态进程立即放弃 CPU
- A. 仅 I、IV B. 仅 II、III C. 仅 I、II、III D. 仅 I、III、IV

I 同时最多一进程进临界区（互斥），II 临界区空闲时允许申请者进入（空闲让进），III 等待进入的时间有界（有限等待），都是临界区互斥机制要求。IV “进不去立即放弃 CPU”是让权等待，可用阻塞实现，但自旋互斥也能在适当条件下保证前面性质，不属此题必须遵循项。I、II、III，选 C。第一笔将安全（不能同进）和推进（空闲要进、不能无限等）分开。

02 与 06 恰好用于排除 IV

Peterson 以自旋等待，可互斥且有界等待；06 的 TSL 会忙等，说明互斥本身不自动要求睡眠。TSL 的简单实现未必保证 III，不能拿它单独证明所有性质；这里排 IV 用的是“让权是另一项额外要求”。

13 | 2024-28：同进程共享 fd，但不能拿别的线程栈顶接着运行

28. 若进程 P 中的线程 T 先打开文件，得到文件描述符 fd，再创建两个线程 Ta 和 Tb，则下列资源中，Ta 与 Tb 可共享的是（ ）。

- I. 进程 P 的地址空间
 - II. 线程 T 的栈
 - III. 文件描述符 fd
- A. 仅 I B. 仅 I、III
- C. 仅 II、III D. I、II、III

Ta、Tb 均由进程 P 创建，共享 P 的地址空间 I，也可访问 P 已打开文件的描述符 fd III；线程 T 的栈 II 是 T 自己的调用与局部执行现场，不能当两新线程的同一栈来共享。仅 I、III，选 B。第一笔沿 0825-07 的“进程资源/线程现场”分类，不被“T 先打开文件”误导为 fd 仅属 T。

“共享地址空间”与“共享栈”措辞差异

线程在同一地址空间技术上可能读到彼此栈内存，但它们的栈作为调用现场须各自维护，题目所说的可共享资源不把 T 的栈列入。fd 是进程打开文件表中的句柄，子线程可使用它；并发读写文件时另需协调文件偏移与数据。

14 | 2024-30：最短的 P 也须轮到第五轮才用满 25ms

30. 假设某系统使用时间片轮转调度算法进行 CPU 调度，时间片大小为 5ms，系统共有 10 个进程，初始时均处于就绪队列，执行结束前仅处于执行态或就绪态。若队尾的进程 P 所需 CPU 时间最短，时间为 25ms，在不考虑系统开销的情况下，则进程 P 的周转时间为（ ）。

- A. 200ms
- B. 205ms
- C. 250ms
- D. 295ms

十个进程起初全就绪，P 在队尾，每片 5ms；P 需 25ms，即 5 片。P 是需 CPU 时间最短者，所以别的九个进程在 P 完成前至少也要运行五片，不会提前退出队列。每轮 10 人各用 5ms，P 分别在 50、100、150、200、250ms 处用完第 1 至 5 片，周转 250ms，选 C。第一笔先问“在 P 第五次上 CPU 前，前面九人会不会消失”。

为什么不是最后一轮少等九人

它们每个 CPU 需求至少 25ms，至 P 第五轮前至多各用了前四片 20ms，仍要各用第五片；队列次序不变，P 第五轮仍排在末尾。 $9 \times 5 \times 5 + 25 = 250\text{ms}$ 是同一计算。

15 | 2025-27：最小页框数从一条指令最多同时需要哪些页推

27. 确定进程运行所需的最少页框数时，要考虑的指标是

- A. 代码段长
- B. 虚拟地址空间大小
- C. 物理地址空间大小
- D. 指令系统支持的寻址方式

题问让进程能运行所需的最少页框数。某条指令本身和它的操作数可能位于不同页，间接寻址等寻址方式还可能在访存中要求额外页面同时可用；决定最低需求的是指令系统支持的寻址方式，选 D。代码段长、虚拟地址空间大与物理地址空间大都不直接说明“一条指令执行时最多得同时留住几页”。第一笔数执行本条指令不能互相替换掉的页面。

为何接在并发节点末尾

若一个进程得到的页框连一条复杂指令所需的页面都容不下，取指、取操作数间反复缺页，根本无法推进；下一节点会进入内存管理。这里只需区分“最低可运行门槛”和“希望少缺页的更大工作集”，不要扩成整章页置换。

15 题收束：从长解释压成几条可执行问句

先问共享状态属于谁；再问读改写是否原子；若要等，问自旋还是阻塞，谁唤醒谁；若要算资源/时间，构造所有人最难推进的状态并核对边界。即使一时做不出标准证明，也能用这些问句排掉“同名即共享”“忙等即让权”“片尽即唤醒”“变量值即资源数”的错误路线。

熟练目标是看到条件就能在 1—2 分钟写出反例或最小可推进状态；这里仅完成 15/15 教学文稿，用户独立计时通关仍待检验。